

COMICCRAFT

AI-Powered Comic Story & Image Generation Platform

Project Documentation

Technology: Python • FastAPI • Gemini • Hugging Face Diffusers • Stable Diffusion • Jinja2

Prepared for project submission and demonstration

1. Project Overview

ComicCraft is an AI-powered comic creation application designed to transform a user's story idea into a multi-panel comic. The application combines a text-generation model for story and panel planning with an image-generation pipeline for visual panel creation. Generated panels can be assembled and exported as a comic PDF.

2. Abstract

ComicCraft simplifies comic creation by allowing a user to provide a character, setting, art style, and story prompt. The backend processes the request, generates a structured comic outline, creates panel images, and prepares an exportable result. The project uses FastAPI for the backend API, Gemini for generative text tasks, Hugging Face/Diffusers for image generation, and a PDF export component for producing the final comic.

3. Objectives

- Provide a simple interface for generating an AI-assisted comic.
- Generate a structured multi-panel story from a user prompt.
- Create visual illustrations for comic panels.
- Return generated panel information through API endpoints.
- Export the generated comic as a PDF.
- Provide a maintainable Python project structure suitable for local development and demonstration.

4. Key Features

- AI-assisted comic story generation.
- Configurable character name, setting, art style, story prompt, and panel count.
- Gemini-based text generation.
- Hugging Face authentication support.
- Stable Diffusion image generation through Diffusers.
- Generated image storage under static/panels.
- PDF comic export.
- FastAPI REST endpoints.
- HTML interface using Jinja2 templates.
- Automated API testing with pytest.
- Demo mode configuration for controlled development/testing.

5. System Architecture

The application follows a backend-centered architecture. A user submits comic parameters through the web interface or API. FastAPI routes validate and process the request. The story-generation service

communicates with Gemini, while the image-generation service uses Hugging Face/Diffusers and Stable Diffusion. Generated images are stored locally and the exporter assembles the comic into a PDF.

Layer	Component	Responsibility
Presentation	Jinja2 / HTML	User-facing comic generation interface
API	FastAPI / routes.py	HTTP endpoints and request handling
Configuration	config.py	Environment variables and application settings
Text AI	Gemini service	Story and panel outline generation
Image AI	image_generator.py	Panel image generation
Storage	static/panels	Generated panel image files
Export	exporters.py	Comic/PDF creation
Testing	pytest	Automated API tests

6. Project Structure

```
ComicCraft/  
├── app/  
│   ├── main.py  
│   ├── routes.py  
│   ├── config.py  
│   └── services/  
│       ├── gemini_flash.py  
│       ├── image_generator.py  
│       └── exporters.py  
├── templates/  
├── static/  
├── panels/  
├── tests/  
│   └── test_api.py  
├── requirements.txt  
└── .env (local configuration; do not publish secrets)
```

7. Configuration

ComicCraft uses Pydantic Settings to load configuration. The project configuration includes Gemini and Hugging Face API keys, Gemini model names, image provider, image model, demo mode, and panel count.

```
gemini_api_key  
hf_api_key  
gemini_model  
gemini_story_model  
image_provider  
image_model  
demo_mode  
panel_count
```

API keys should be stored locally and never committed to a public GitHub repository. A .env file is normally used for local secrets when the application configuration enables env-file loading.

8. AI Text Generation

The Gemini service converts the user's story idea into structured comic content. The generated outline can contain panel descriptions and other information required by the image-generation stage. The configured story model and general Gemini model are controlled through application settings.

9. AI Image Generation

The image generator uses Hugging Face authentication and the Diffusers library. The current configuration shown during development uses the Hugging Face provider and the model `stabilityai/stable-diffusion-xl-base-1.0`. The image generator loads the pipeline, generates panel artwork from prompts, and stores PNG files under `static/panels`.

- Checks whether CUDA is available and otherwise uses CPU.
- Builds a comic-focused prompt from the panel description.
- Uses a negative prompt to reduce common visual defects.
- Generates an image using Diffusers.
- Saves the image as a PNG file.
- Returns a static URL for the FastAPI/Jinja2 application.

10. PDF Export

The exporter component combines generated comic panels and related content into a PDF output. The resulting PDF URL is returned by the API so that the generated comic can be opened or downloaded from the application.

11. API Endpoints

Endpoint	Method	Purpose
/	GET	Open the ComicCraft home page
/generate-comic/json	POST	Generate a comic from JSON input
/generate	POST	Generate a comic from form input

12. Testing

The project uses `pytest` for API testing. During development, the test suite was run with `python -m pytest -q`. The project reached a successful state with 6 tests passing. Earlier failures were related to missing dependencies, image-model loading, and configuration issues; these were addressed during development.

```
python -m pytest -q
```

13. Installation and Setup

- Open PowerShell in the ComicCraft project directory.
- Use Python's module form of `pip` when the `pip` command is not on `PATH`.
- Install dependencies from `requirements.txt`.

- Configure the required API keys locally.
- Start the FastAPI application with Python's module form of Uvicorn if the uvicorn command is not on PATH.
- Open the local application in a browser and test comic generation.

```
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --reload
```

14. Security and Secret Management

- Never commit Gemini or Hugging Face API keys to GitHub.
- Keep .env local and add it to .gitignore.
- If a secret is accidentally published, revoke/rotate the key immediately.
- Use environment-based configuration for deployment.

15. Known Development Notes

- The Windows environment may not have the Scripts directory on PATH; python -m pip and python -m uvicorn avoid this issue.
- Stable Diffusion models can require substantial RAM/VRAM and may take several minutes to load on CPU.
- Deprecation warnings from Starlette/AnyIO or template response APIs are warnings rather than test failures.
- Image generation depends on the configured model, Hugging Face access, available hardware, and network/cache availability.

16. Future Enhancements

- Add user accounts and saved comic projects.
- Add persistent database storage.
- Add more art-style presets.
- Add image regeneration for individual panels.
- Add panel editing and reordering.
- Add cloud deployment and object storage.
- Add asynchronous/background image generation for better responsiveness.
- Add richer PDF layouts and cover-page customization.

17. Conclusion

ComicCraft demonstrates an end-to-end generative AI workflow in which a natural-language story idea is transformed into structured comic content, illustrated panels, and a PDF comic. The combination of FastAPI, Gemini, Hugging Face Diffusers, Stable Diffusion, and automated testing provides a practical foundation for an AI-assisted creative application.

18. Project Demonstration Checklist

- Start the FastAPI application.
- Open the ComicCraft home page.
- Enter a character name, setting, art style, and story prompt.
- Generate the comic.
- Verify the generated panels.
- Open the generated PDF.
- Show the API documentation if required.
- Show the successful pytest result.